

Problem1

May 31, 2026

0.1 4.1 Obstacle Avoidance

(a)

```
[ ]: def signed_distances(s, centers, radii):  
    """Compute signed distances to circular obstacles."""  
    centers = jnp.reshape(centers, (-1, 2))  
    radii = jnp.reshape(radii, (-1,))  
  
    d = jnp.linalg.norm(s[:2] - centers, axis=1) - radii  
  
    return d
```

(b)

```
[ ]: @partial(jax.jit, static_argnums=(0,))  
    @partial(jax.vmap, in_axes=(None, 0, 0))  
    def affinize(f, s, u):  
        """Affinize the function `f(s, u)` around `(s, u)`."""  
        A, B = jax.jacobian(f, argnums=(0, 1))(s, u)  
        c = f(s, u) - A @ s - B @ u  
        return A, B, c
```

(c)

Linearizing f and d around the current iterate $(s_{k|t}^{(i)}, u_{k|t}^{(i)})$:

$$A_{f,k|t}^{(i)} = \frac{\partial f}{\partial s} \Big|_{s_{k|t}^{(i)}, u_{k|t}^{(i)}}, \quad B_{f,k|t}^{(i)} = \frac{\partial f}{\partial u} \Big|_{s_{k|t}^{(i)}, u_{k|t}^{(i)}}, \quad c_{f,k|t}^{(i)} = f(s_{k|t}^{(i)}, u_{k|t}^{(i)}) - A_{f,k|t}^{(i)} s_{k|t}^{(i)} - B_{f,k|t}^{(i)} u_{k|t}^{(i)}$$

$$A_{d,k|t}^{(i)} = \frac{\partial d}{\partial s} \Big|_{s_{k|t}^{(i)}}, \quad c_{d,k|t}^{(i)} = d(s_{k|t}^{(i)}) - A_{d,k|t}^{(i)} s_{k|t}^{(i)}$$

The convex sub-problem is:

$$\begin{aligned}
& \min_{s_{\cdot|t}, u_{\cdot|t}} (s_{N|t} - s_{\text{goal}})^\top P (s_{N|t} - s_{\text{goal}}) + \sum_{k=0}^{N-1} [(s_{k|t} - s_{\text{goal}})^\top Q (s_{k|t} - s_{\text{goal}}) + u_{k|t}^\top R u_{k|t}] \\
& \text{s.t. } s_{0|t} = s(t) \\
& \quad s_{k+1|t} = A_{f,k|t}^{(i)} s_{k|t} + B_{f,k|t}^{(i)} u_{k|t} + c_{f,k|t}^{(i)}, \quad k = 0, \dots, N-1 \\
& \quad A_{d,k|t}^{(i)} s_{k|t} + c_{d,k|t}^{(i)} \geq 0, \quad k = 0, \dots, N
\end{aligned}$$

(d)

Each $d_j(s) = \|(x, y) - (x_j, y_j)\|_2 - r_j$ is convex (norm minus a constant). For any convex function g , the first-order condition gives $g(x) \geq g(\bar{x}) + \nabla g(\bar{x})^\top (x - \bar{x})$ for all x . Rearranging, $g(x) \geq \underbrace{\nabla g(\bar{x})^\top x}_{A_d} + \underbrace{g(\bar{x}) - \nabla g(\bar{x})^\top \bar{x}}_{c_d}$, so

$$d(s_{k|t}) \geq A_{d,k|t}^{(i)} s_{k|t} + c_{d,k|t}^{(i)}$$

Therefore $A_{d,k|t}^{(i)} s_{k|t} + c_{d,k|t}^{(i)} \geq 0 \implies d(s_{k|t}) \geq 0$. The linearized constraint is a conservative inner approximation of the true obstacle constraint, so feasibility of the sub-problem guarantees safety.

(e)

```
[ ]: def scp_iteration(f, d, s0, s_goal, s_prev, u_prev, P, Q, R):
    """Solve a single SCP sub-problem for the obstacle avoidance problem."""
    n = s_prev.shape[-1]
    m = u_prev.shape[-1]
    N = u_prev.shape[0]

    Af, Bf, cf = affinize(f, s_prev[:-1], u_prev)
    Af, Bf, cf = np.array(Af), np.array(Bf), np.array(cf)
    Ad, _, cd = affinize(
        lambda s, _: d(s), s_prev, jnp.concatenate((u_prev, u_prev[-1:]))
    )
    Ad, cd = np.array(Ad), np.array(cd)

    s_cvx = cvx.Variable((N + 1, n))
    u_cvx = cvx.Variable((N, m))

    objective = 0.0
    constraints = [s_cvx[0] == s0]

    for k in range(N):
        objective += cvx.quad_form(s_cvx[k] - s_goal, Q) + cvx.
        ↪quad_form(u_cvx[k], R)
        constraints += [s_cvx[k + 1] == Af[k] @ s_cvx[k] + Bf[k] @ u_cvx[k] +
        ↪cf[k]]
        constraints += [Ad[k] @ s_cvx[k] + cd[k] >= 0]
```

```

objective += cvx.quad_form(s_cvx[N] - s_goal, P)
constraints += [Ad[N] @ s_cvx[N] + cd[N] >= 0]

prob = cvx.Problem(cvx.Minimize(objective), constraints)
prob.solve()
s = s_cvx.value
u = u_cvx.value
J = prob.objective.value
return s, u, J

```

(f)

```

[ ]: # Solve the MPC problem at time `t`
s_mpc[t], u_mpc[t] = solve_obstacle_avoidance_scp(
    f, d, s, s_goal, N, P, Q, R, eps, N_scp, s_init=s_init, u_init=u_init
)

# Push the state `s` forward in time with a closed-loop MPC input
s = np.array(f(s, u_mpc[t, 0]))

```

(g)

N	N_{SCP}	total_time (s)	total_control_cost
5	5	13.91	6.93
2	5	13.08	11.18
5	2	8.45	4.43
15	2	11.49	4.50

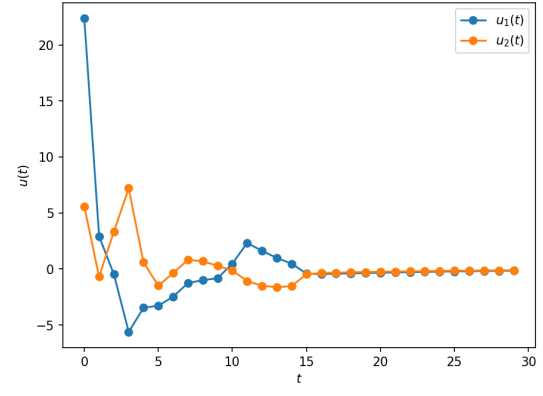
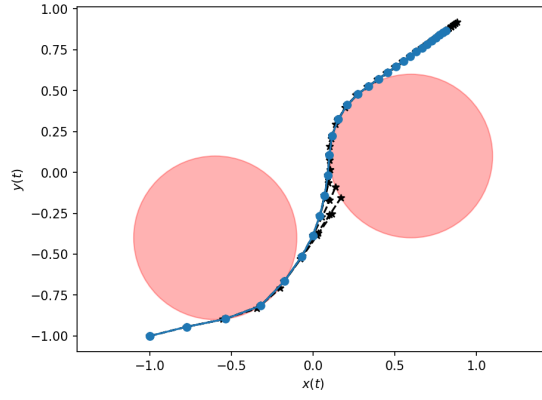
```

[1]: from IPython.display import display, Image

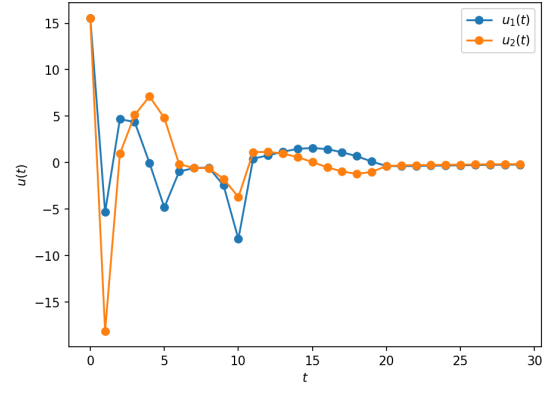
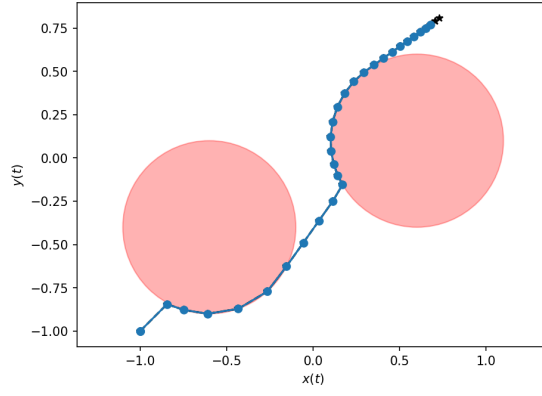
for fname in [
    "../soln_obstacle_avoidance_N=5_Nscp=5.png",
    "../soln_obstacle_avoidance_N=2_Nscp=5.png",
    "../soln_obstacle_avoidance_N=5_Nscp=2.png",
    "../soln_obstacle_avoidance_N=15_Nscp=2.png",
]:
    display(Image(filename=fname))

```

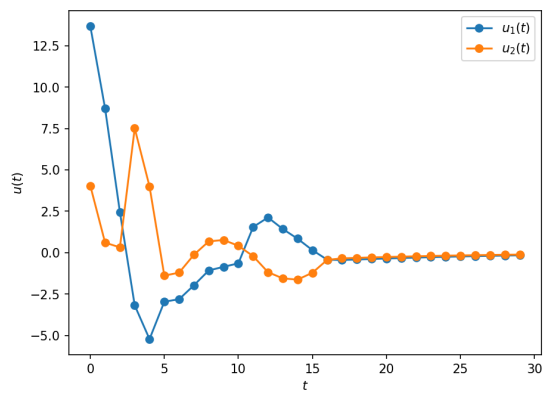
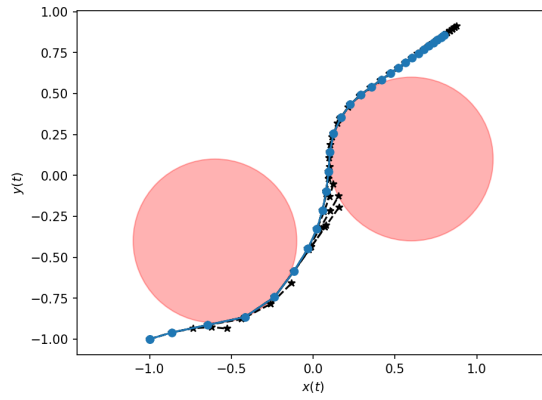
$N = 5, N_{SCP} = 5$

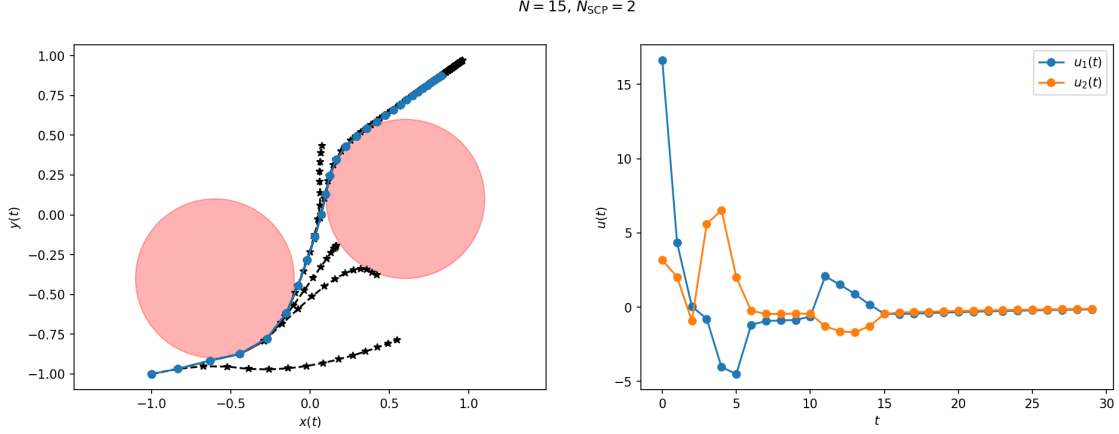


$N = 2, N_{SCP} = 5$



$N = 5, N_{SCP} = 2$





All four cases successfully steer the system from $(-1, -1)$ toward $(1, 1)$ while avoiding both obstacles.

With $N = 5$ and $N_{\text{SCP}} = 5$, the trajectory smoothly navigates between the two obstacles, though the control inputs show a large initial spike in u_1 before settling, resulting in a cost of 6.93.

Reducing the horizon to $N = 2$ (with $N_{\text{SCP}} = 5$) makes the controller myopic, since it cannot anticipate the obstacles early enough. As a result, the trajectory hugs the obstacle boundaries more tightly and the controls become much more aggressive and oscillatory, which explains the highest control cost of 11.18.

With $N = 5$ and $N_{\text{SCP}} = 2$, the trajectory is very similar to the $N = 5, N_{\text{SCP}} = 5$ case but with smaller control magnitudes. This is the fastest run (8.45s) and achieves the lowest cost (4.43), suggesting that SCP already converges in just 2 iterations here.

Increasing the horizon to $N = 15$ (with $N_{\text{SCP}} = 2$) allows the planner to see further ahead, as shown by the longer planned trajectories. However, the actual trajectory and control cost (4.50) are comparable to the $N = 5, N_{\text{SCP}} = 2$ case, while the computation takes longer (11.49s) because of the larger optimization problem. Overall, a moderate horizon with few SCP iterations provides the best trade-off between computation time and trajectory quality.

Problem2

May 31, 2026

0.1 4.2 Widget Sales

(a)

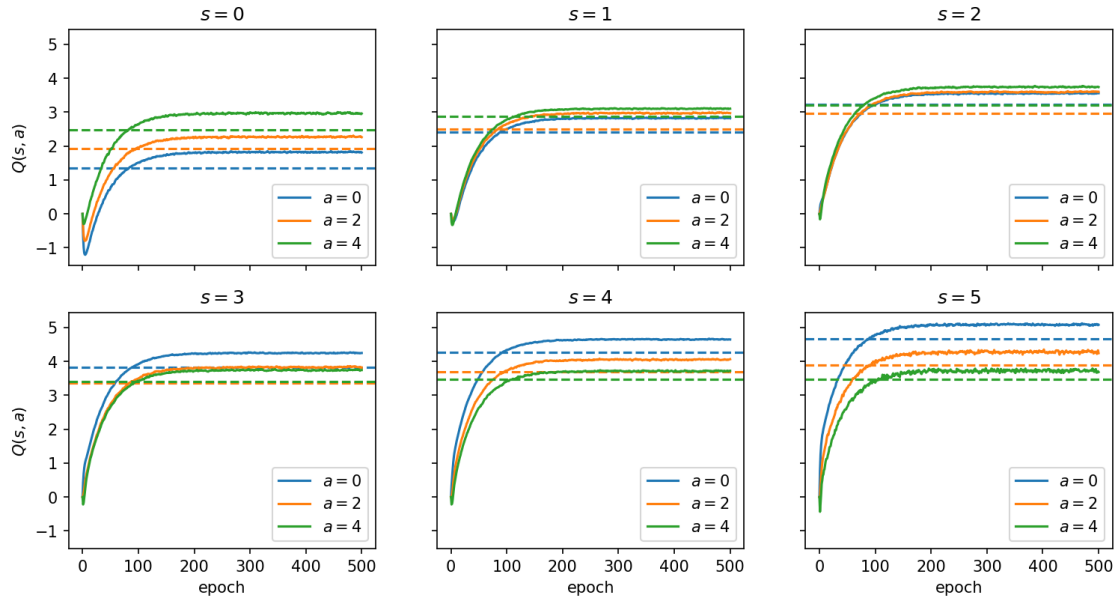
```
[ ]: for idx in shuffled_indices:
    s_t = int(log["s"][idx])
    a_t = int(log["a"][idx])
    r_t = log["r"][idx]
    s_next = int(log["s"][idx + 1])

    a_idx = np.searchsorted(A, a_t)
    Q[s_t, a_idx] += * (r_t + * np.max(Q[s_next]) - Q[s_t, a_idx])
```

(b)

```
[ ]: Q_vi_new = np.zeros_like(Q_vi)
    for i, s in enumerate(S):
        for j, a in enumerate(A):
            Q_vi_new[i, j] = sum(
                P[l] * (reward(s, a, d) + * np.max(Q_vi[transition(s, a, d)]))
                for l, d in enumerate(D)
            )
    Q_vi_prev = Q_vi.copy()
    Q_vi[:] = Q_vi_new
```

```
[1]: from IPython.display import display, Image
    display(Image(filename="../widget_sales_qvalues.png"))
```



The Q-learning values (solid lines) converge toward the value iteration values (dashed lines) but do not match them exactly. For states $s = 0, 1, 4, 5$ the action ranking is the same between both methods, so the optimal action does not change. However, at $s = 2$ the ranking differs: Q-learning ranks $a = 4 > a = 2 > a = 0$ while value iteration gives $a = 0 > a = 4 > a = 2$, leading to a different optimal action ($a = 4$ vs $a = 0$). At $s = 3$ the best action agrees ($a = 0$) but the ordering of the suboptimal actions is swapped. These differences arise because Q-learning learns from real transitions while value iteration relies on the intern's approximate model of the dynamics and demand distribution.

(c)

```
[ ]: a_opt_ql = A[np.argmax(Q, axis=1)]
     a_opt_vi = A[np.argmax(Q_vi, axis=1)]

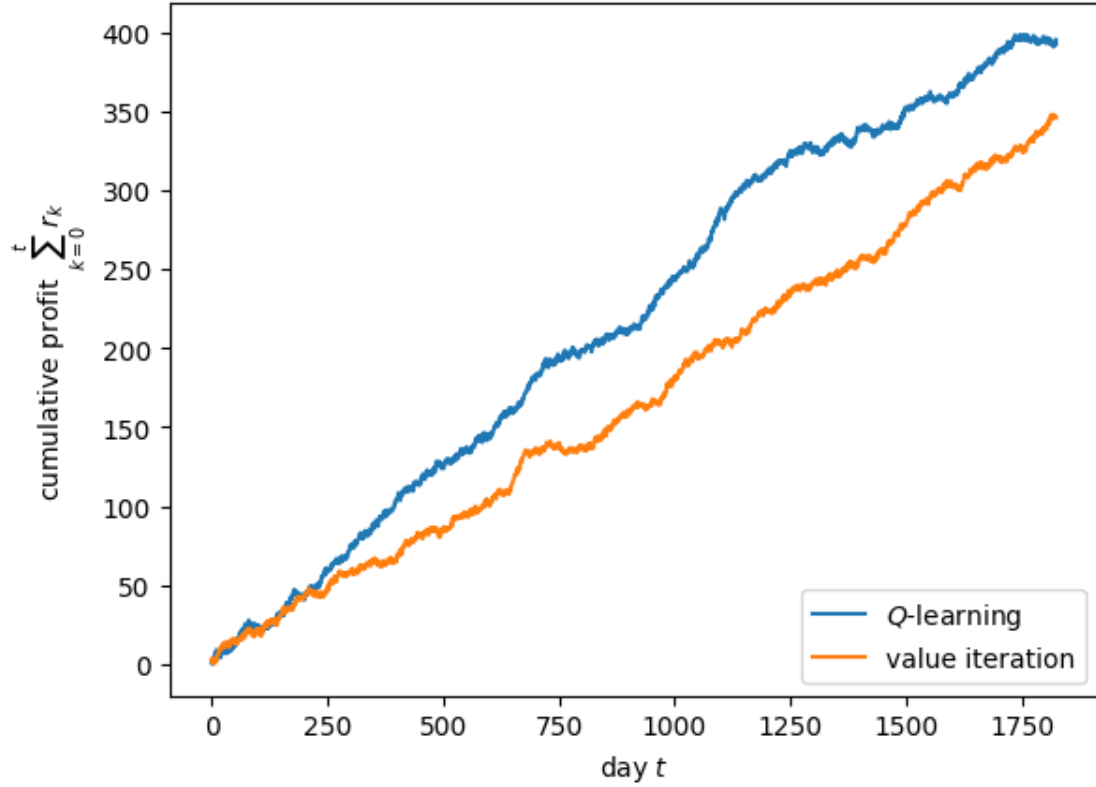
_, _, r_ql = simulate(rng, lambda s: a_opt_ql[int(s)], T)
profit_ql = np.cumsum(r_ql)

_, _, r_vi = simulate(rng, lambda s: a_opt_vi[int(s)], T)
profit_vi = np.cumsum(r_vi)
```

Optimal policy (Q-learning): $[4, 4, 4, 0, 0, 0]$

Optimal policy (value iteration): $[4, 4, 0, 0, 0, 0]$

```
[2]: display(Image(filename="../widget_sales_profits.png"))
```



The Q-learning policy accumulates higher profit than the value iteration policy over 5 years. The two policies differ only at $s = 2$: Q-learning orders 4 widgets while value iteration orders 0. Since the intern's model is only an approximation of the true dynamics and demand distribution, value iteration is optimal with respect to that approximate model but not necessarily with respect to reality. Q-learning, on the other hand, learned directly from real historical transitions, so it may have captured the true underlying dynamics more accurately. The higher cumulative profit of Q-learning suggests that the model errors in the intern's approximation hurt the value iteration policy, particularly at $s = 2$ where the two methods disagree.

Problem3

May 31, 2026

0.1 4.3 CartPole

(a)

```
[ ]: def build_network(self) -> torch.nn.Module:
    Q=torch.nn.Sequential(
        torch.nn.Linear(self.state_dim, self.hidden_dim),
        torch.nn.ReLU(),
        torch.nn.Linear(self.hidden_dim, self.hidden_dim),
        torch.nn.ReLU(),
        torch.nn.Linear(self.hidden_dim, self.action_dim)
    )
    return Q

[ ]: def policy(self, state : Union[np.ndarray, torch.tensor], train : bool=False)
    ↪-> torch.Tensor:
    if isinstance(state, np.ndarray):
        state = torch.tensor(state).unsqueeze(0).to(self.device)
    if train:
        if random.random()<self.eps_threshold():
            return torch.randint(0, self.action_dim, (1,)).to(self.device)
        else:
            with torch.no_grad():
                return torch.argmax(self.policy_network(state)).unsqueeze(0)
    else:
        with torch.no_grad():
            return torch.argmax(self.policy_network(state)).unsqueeze(0)

[ ]: def train(self, env : gym.wrappers, num_episodes : int=100) -> None:
    optimizer=torch.optim.AdamW(self.policy_network.parameters(), lr=self.
    ↪learning_rate)
    loss_fn=torch.nn.MSELoss()
    for episode in tqdm(range(num_episodes)):
        state, _ = env.reset()
        done = False
        while not done:
            action=self.policy(state, train=True)
```

```

        next_state, reward, terminated, truncated, _ = env.step(action.
↪item())
        done=terminated or truncated
        sp = None if terminated else torch.tensor(next_state).unsqueeze(0).
↪to(self.device)
        self.buffer.append(Transition(torch.tensor(state).unsqueeze(0).
↪to(self.device), action, torch.tensor(reward).unsqueeze(0).to(self.device),
↪sp))
        state=next_state

        if len(self.buffer)>self.batch_size:
            states, actions, targets=self.sample_buffer()
            predictions=self.policy_network(states).gather(1, actions)
            loss=loss_fn(predictions, targets)
            optimizer.zero_grad()
            loss.backward()
            torch.nn.utils.clip_grad_value_(self.policy_network.
↪parameters(), 100)
            optimizer.step()

```

DQN evaluation on CartPole-v1: - Before training: average reward = 11.4 - After training (500 episodes): average reward = 500.0

(b)

```

[ ]: def learn(self, rewards: list, log_probs: list) -> None:
    T = len(rewards)
    log_probs = torch.stack(log_probs)

    # 1) Naive REINFORCE
    # R = sum(self.gamma**t * rewards[t] for t in range(T))
    # loss = -(log_probs * R).sum()

    # 2) REINFORCE with causality trick
    # returns = torch.zeros(T)
    # for t in range(T):
    #     returns[t] = sum(self.gamma**(tp - t) * rewards[tp] for tp in
↪range(t, T))
    # loss = -(log_probs * returns).sum()

    # 3) REINFORCE with causality trick and baseline to "center" the returns
    returns = torch.zeros(T)
    for t in range(T):
        returns[t] = sum(self.gamma**(tp - t) * rewards[tp] for tp in range(t,
↪T))
    baseline = sum(self.gamma**t * rewards[t] for t in range(T)) / T
    loss = -(log_probs * (returns - baseline)).sum()

```

```
[ ]: from IPython.display import display, Image

print("Naive REINFORCE:")
display(Image(filename="../reward_curve_naive.png"))
print("REINFORCE with causality trick:")
display(Image(filename="../reward_curve_causality.png"))
print("REINFORCE with causality trick + baseline:")
display(Image(filename="../reward_curve_baseline.png"))
```

DQN is more sample efficient than REINFORCE because it stores transitions in a replay buffer and performs a gradient step at every environment step, reusing past experience multiple times. REINFORCE, on the other hand, collects trajectories online with the current policy and discards them after a single gradient update, making it inherently less data efficient. As a result, DQN reaches an average reward of 500.0 after 500 episodes while all three REINFORCE variants achieve lower rewards.

Among the REINFORCE variants, the causality trick improves learning since the variance from past rewards is removed from the gradient computation, allowing each action to be reinforced only based on its future consequences. The baseline variant further removes state-relative variance for a similar final performance as the causality trick.

```
[ ]:
```

Problem4

May 31, 2026

0.1 4.4 Learning LQR

(a)

Model-based RLS assumes linear dynamics $x_{t+1} = Ax_t + Bu_t$ and quadratic cost $c(x, u) = x^\top Qx + u^\top Ru$. It explicitly learns estimates $\hat{A}, \hat{B}, \hat{Q}, \hat{R}$ from data via recursive least-squares, then solves the Riccati equation to obtain the optimal gain K . This method exploits the most problem structure and converges in roughly 10 episodes, as seen in Figure 1.

Model-free policy iteration assumes the Q-function has a quadratic structure $Q_K(x, u) = (x, u)^\top H_K(x, u)$, but does not learn a dynamics model. It fits H_K directly from observed temporal-difference errors using least-squares, then updates the policy as $K \leftarrow -H_{K,22}^{-1}H_{K,12}^\top$. Since it avoids estimating A, B and only relies on the quadratic Q-function assumption, it uses less structural information than model-based RLS and converges in roughly 1000 episodes.

Model-free policy gradients makes no structural assumptions about the dynamics or the cost. It treats the policy as a parametric black box $\pi_K(u | x) = \mathcal{N}(Kx, \Sigma)$ and updates K via Monte Carlo REINFORCE gradients. Because it does not exploit any problem structure, it requires roughly 10,000 episodes and still has not fully converged.

Overall, the more structural assumptions a method incorporates, the faster and more precisely it converges. Model-based RLS reaches the smallest gain error $\|K^{(i)} - K^*\|_F$ and the lowest cost, while policy gradients retains a significant residual error even after 10,000 episodes. In this case, the true underlying system is indeed linear with quadratic cost, so exploiting these assumptions does not introduce any model mismatch or bias. Had the true system been nonlinear or the cost non-quadratic, the stronger assumptions of model-based RLS and policy iteration would have introduced a bias, potentially making the assumption-free policy gradient method more robust despite its slower convergence.

(b)

Model-based RLS can most naturally incorporate prior beliefs on A, B, Q, R through regularization in the least-squares fits, which corresponds to Bayesian regression with a prior. For instance, a Gaussian prior on A, B translates directly into a ridge penalty centered at the prior mean, biasing estimates toward the prior especially when data is scarce.

Model-free policy iteration does not estimate A, B directly, so priors on the dynamics cannot be incorporated in a straightforward way. However, prior beliefs on the cost matrices Q, R could inform the initialization or regularization of the H_K fit, since H_K implicitly depends on Q and R .

Model-free policy gradients has no internal model to regularize, making it the hardest method in which to incorporate prior beliefs. At best, prior knowledge of A, B could be used to compute a

good initial policy $K^{(0)}$ (e.g., via Riccati on the prior mean), and prior knowledge of cost magnitudes could help set the learning rate or standardization parameters.

Problem5

May 31, 2026

0.1 4.5 Lunar Lander

(a)

The code computes the advantage estimate as:

```
advantages = returns - values
```

where `returns` are the Monte Carlo tail returns $G_t = \sum_{\tau=t}^T \gamma^{\tau-t} r_\tau$ computed from a single episode rollout via `compute_returns`, and `values` = $V_w^\pi(x_t)$ are the critic's learned estimates from the `value_head` of the shared network. This corresponds to the advantage estimator:

$$\delta^\pi = G_t - V_w^\pi(x_t)$$

On the bias-variance spectrum, this estimator is **low bias, high variance**. It is unbiased since G_t is a Monte Carlo estimate of $Q^\pi(x_t, u_t)$, so the advantage estimate is an unbiased estimate of $A^\pi(x_t, u_t)$. However, it has high variance since it relies on the full trajectory return, which accumulates randomness across all timesteps. Subtracting the baseline $V_w^\pi(x_t)$ reduces variance without introducing bias, because the baseline depends only on the state and not on the action, so it vanishes in expectation when computing the policy gradient.

(b)

Returns can vary by orders of magnitude across training, as early episodes may yield very negative returns while later ones are large and positive. Without standardization, gradient magnitudes would be inconsistent across episodes, making learning unstable. The code standardizes returns using a running EMA of the mean and variance:

```
standardized_returns = (returns - return_ema) / (np.sqrt(return_emv) + 1e-6)
```

This keeps the critic targets and actor gradient weights at a consistent scale throughout training, similar in spirit to batch normalization, and improves learning stability.

(c)

`jax.lax.stop_gradient` prevents gradients from flowing through the advantage estimate when computing the actor loss:

```
actor_loss = jnp.sum(-action_log_probs * jax.lax.stop_gradient(advantages) * mask)
```

Without it, the gradient of `actor_loss` would flow through `advantages = returns - values`, causing the actor update to also push the critic's `values` toward `returns`, effectively mixing actor and critic gradients through the shared trunk. This is incorrect: the policy gradient theorem

requires the advantage to be treated as a fixed scalar weight δ^π when differentiating with respect to θ . The critic already learns separately through `critic_loss = sum(advantages^2)`, so `stop_gradient` ensures the two losses remain decoupled as intended.

(d)

An alternative approach would be model-based trajectory optimization, such as iLQR or SCP (as in Problem 1). If the lunar lander dynamics were known or could be identified from data, one could linearize the dynamics at each timestep and solve a finite-horizon optimal control problem to compute a control sequence. Compared to the model-free A2C approach, trajectory optimization is significantly more sample efficient and can provide stronger guarantees on the quality of the solution. However, it requires an accurate dynamics model, since lunar lander contact forces and thruster dynamics are difficult to model precisely and model mismatch would introduce suboptimality or instability. The model-free RL approach here makes no assumptions on the dynamics and is more general, at the cost of requiring thousands of environment interactions to converge.

obstacle_avoidance.py

```

"""
Starter code for the problem "Obstacle avoidance".

Autonomous Systems Lab (ASL), Stanford University
"""

from funtools import partial
from time import time

import cvxpy as cvx

import jax
import jax.numpy as jnp

import matplotlib.pyplot as plt

import numpy as np

from tqdm.auto import tqdm

def dynamics(s, u, beta=0.3, dt=0.1):
    """Evaluate the discrete-time dynamics of the system."""
    _, _, dx, dy = s
    ds = jnp.array(
        [
            dx,
            dy,
            u[0] - beta * dx * jnp.abs(dx),
            u[1] - beta * dy * jnp.abs(dy),
        ]
    )
    s_next = s + dt * ds
    return s_next

def signed_distances(s, centers, radii):
    """Compute signed distances to circular obstacles."""
    centers = jnp.reshape(centers, (-1, 2))
    radii = jnp.reshape(radii, (-1,))

    # PART (a): YOUR CODE BELOW #####
    # INSTRUCTIONS: Compute the vector of signed distances to each obstacle.

    d = jnp.linalg.norm(s[:2] - centers, axis=1) - radii

    # END PART (a) #####
    return d

@partial(jax.jit, static_argnums=(0,))
@partial(jax.vmap, in_axes=(None, 0, 0))
def affinize(f, s, u):
    """Affinize the function `f(s, u)` around `(s, u)`."""
    # PART (b) #####
    # INSTRUCTIONS: Use JAX to affinize `f` around `(s,u)` in two lines.

    A, B = jax.jacobian(f, argnums=(0, 1))(s, u)
    c = f(s, u) - A @ s - B @ u

    # END PART (b) #####
    return A, B, c

def scp_iteration(f, d, s0, s_goal, s_prev, u_prev, P, Q, R):
    """Solve a single SCP sub-problem for the obstacle avoidance problem."""
    n = s_prev.shape[-1] # state dimension
    m = u_prev.shape[-1] # control dimension
    N = u_prev.shape[0] # number of steps

    Af, Bf, cf = affinize(f, s_prev[:-1], u_prev)
    Af, Bf, cf = np.array(Af), np.array(Bf), np.array(cf)
    Ad, _, cd = affinize(
        lambda s, _: d(s), s_prev, jnp.concatenate((u_prev, u_prev[-1:]))
    )
    Ad, cd = np.array(Ad), np.array(cd)

    s_cvx = cvx.Variable((N + 1, n))
    u_cvx = cvx.Variable((N, m))

```

```

# PART (e): YOUR CODE BELOW #####
# INSTRUCTIONS: Construct the convex SCP sub-problem.

objective = 0.0
constraints = [s_cvx[0] == s0]

for k in range(N):
    objective += cvx.quad_form(s_cvx[k] - s_goal, Q) + cvx.quad_form(u_cvx[k], R)
    constraints += [s_cvx[k + 1] == Af[k] @ s_cvx[k] + Bf[k] @ u_cvx[k] + cf[k]]
    constraints += [Ad[k] @ s_cvx[k] + cd[k] >= 0]

objective += cvx.quad_form(s_cvx[N] - s_goal, P)
constraints += [Ad[N] @ s_cvx[N] + cd[N] >= 0]

# END PART (e) #####

prob = cvx.Problem(cvx.Minimize(objective), constraints)
prob.solve()
if prob.status != "optimal":
    raise RuntimeError("SCP solve failed. Problem status: " + prob.status)
s = s_cvx.value
u = u_cvx.value
J = prob.objective.value
return s, u, J

def solve_obstacle_avoidance_scp(
    f,
    d,
    s0,
    s_goal,
    N,
    P,
    Q,
    R,
    eps,
    max_iters,
    s_init=None,
    u_init=None,
    convergence_error=False,
):
    """Solve the obstacle avoidance problem via SCP."""
    n = Q.shape[0] # state dimension
    m = R.shape[0] # control dimension

    # Initialize trajectory
    if s_init is None or u_init is None:
        s = np.zeros((N + 1, n))
        u = np.zeros((N, m))
        s[0] = s0
        for k in range(N):
            s[k + 1] = f(s[k], u[k])
    else:
        s = np.copy(s_init)
        u = np.copy(u_init)

    # Do SCP until convergence or maximum number of iterations is reached
    converged = False
    J = np.zeros(max_iters + 1)
    J[0] = np.inf
    for i in range(max_iters):
        s, u, J[i + 1] = scp_iteration(f, d, s0, s_goal, s, u, P, Q, R)
        dJ = np.abs(J[i + 1] - J[i])
        if dJ < eps:
            converged = True
            break
    if not converged and convergence_error:
        raise RuntimeError("SCP did not converge!")
    return s, u

# Define constants
n = 4 # state dimension
m = 2 # control dimension
s0 = np.array([-1.0, -1.0, 0.0, 0.0]) # initial state
s_goal = np.array([1.0, 1.0, 0.0, 0.0]) # desired final state
T = 30 # total simulation time
P = 1e2 * np.eye(n) # terminal state cost matrix
Q = 1e1 * np.eye(n) # state cost matrix
R = 1e-2 * np.eye(m) # control cost matrix
eps = 1e-3 # SCP convergence tolerance

# Set obstacle center points and radii
centers = np.array(
    [
        [-0.6, -0.4],
        [0.6, 0.1],
    ]
)

```

```

)
radii = np.array([0.5, 0.5])

# PART (g): CHANGE THE PARAMETERS BELOW #####

N = 5 # MPC horizon
N_scp = 5 # maximum number of SCP iterations

# END PART (g) #####

f = dynamics
d = partial(signed_distances, centers=centers, radii=radii)
s_mpc = np.zeros((T, N + 1, n))
u_mpc = np.zeros((T, N, m))
s = np.copy(s0)
total_time = time()
total_control_cost = 0.0
s_init = None
u_init = None
for t in tqdm(range(T)):
    # PART (f): YOUR CODE BELOW #####

    # Solve the MPC problem at time `t`
    s_mpc[t], u_mpc[t] = solve_obstacle_avoidance_scp(
        f, d, s, s_goal, N, P, Q, R, eps, N_scp, s_init=s_init, u_init=u_init
    )

    # Push the state `s` forward in time with a closed-loop MPC input
    s = np.array(f(s, u_mpc[t, 0]))

    # END PART (f) #####

    # Accumulate the actual control cost
    total_control_cost += u_mpc[t, 0].T @ R @ u_mpc[t, 0]

    # Use this solution to warm-start the next iteration
    u_init = np.concatenate([u_mpc[t, 1:], u_mpc[t, -1:]])
    s_init = np.concatenate(
        [s_mpc[t, 1:], f(s_mpc[t, -1], u_mpc[t, -1]).reshape([1, -1])]
    )
total_time = time() - total_time
print("Total elapsed time:", total_time, "seconds")
print("Total control cost:", total_control_cost)

fig, ax = plt.subplots(1, 2, dpi=150, figsize=(15, 5))
fig.suptitle("$N = {}$, ".format(N) + r"$N\_mathrm{SCP} = " + "{}$".format(N_scp))

for pc, rc in zip(centers, radii):
    ax[0].add_patch(plt.Circle((pc[0], pc[1]), rc, color="r", alpha=0.3))
for t in range(T):
    ax[0].plot(s_mpc[t, :, 0], s_mpc[t, :, 1], "--*", color="k")
ax[0].plot(s_mpc[:, 0, 0], s_mpc[:, 0, 1], "-o")
ax[0].set_xlabel(r"$x(t)$")
ax[0].set_ylabel(r"$y(t)$")
ax[0].axis("equal")

ax[1].plot(u_mpc[:, 0, 0], "-o", label=r"$u_1(t)$")
ax[1].plot(u_mpc[:, 0, 1], "-o", label=r"$u_2(t)$")
ax[1].set_xlabel(r"$t$")
ax[1].set_ylabel(r"$u(t)$")
ax[1].legend()

suffix = "_N={}_Nscp={}".format(N, N_scp)
plt.savefig("soln_obstacle_avoidance" + suffix + ".png", bbox_inches="tight")
plt.show()

```

widget_sales.py

```

"""
Starter code for the problem "Widget sales".

Autonomous Systems Lab (ASL), Stanford University
"""

import numpy as np
import matplotlib.pyplot as plt
from tqdm.auto import tqdm

# Seed RNG for reproducibility
seed = 0
rng = np.random.default_rng(seed)

# Define the state space, action space, and demand distribution
S = np.array([0, 1, 2, 3, 4, 5])
A = np.array([0, 2, 4])
D = np.array([0, 1, 2, 3, 4])
P = np.array([0.1, 0.3, 0.3, 0.2, 0.1])

def transition(s: int, a: int, d: int) -> int:
    """Compute the next state given the current state, action, and demand."""
    s_next = np.clip(s + a - d, 0, 5)
    return s_next

def reward(s: int, a: int, d: int) -> float:
    """Compute the reward given the current state, action, and demand."""
    price = 1.2
    cost_rent = 1.0
    cost_storage = 0.05 * s
    cost_order = np.sqrt(a)
    r = price * np.minimum(s + a, d) - cost_rent - cost_storage - cost_order
    return r

def simulate(
    rng: np.random.Generator,
    policy: callable,
    T: int,
    s0: int = 5,
    D: np.ndarray = D,
    P: np.ndarray = P,
) -> tuple[np.ndarray, np.ndarray, np.ndarray]:
    """Simulate widget sales for a given policy."""
    s = np.zeros(T + 1) # states
    a = np.zeros(T) # actions
    r = np.zeros(T) # rewards
    s[0] = s0 # initial state

    for t in tqdm(range(T)):
        # Sample demand
        d = rng.choice(D, p=P)

        # Record action, reward, and next state
        a[t] = policy(s[t])
        r[t] = reward(s[t], a[t], d)
        s[t + 1] = transition(s[t], a[t], d)

    return s, a, r

# Generate historical data with a uniformly random policy
log = {}
T = 3 * 365
log["s"], log["a"], log["r"] = simulate(rng, lambda s, A=A: rng.choice(A), T)

# Do Q-learning
gamma = 0.95 # discount factor
alpha = 1e-2 # learning rate
num_epochs = 5 * int(1 / alpha) # number of epochs

Q = np.zeros((S.size, A.size))
Q_epoch = np.zeros((num_epochs + 1, S.size, A.size))

for k in tqdm(range(1, num_epochs + 1)):
    # Shuffle transition tuple indices

```

```

shuffled_indices = rng.permutation(T)

# ##### PART (a): YOUR CODE BELOW #####

for idx in shuffled_indices:
    s_t = int(log["s"][idx])
    a_t = int(log["a"][idx])
    r_t = log["r"][idx]
    s_next = int(log["s"][idx + 1])

    a_idx = np.searchsorted(A, a_t)
    Q[s_t, a_idx] +=  $\alpha$  * (r_t +  $\gamma$  * np.max(Q[s_next]) - Q[s_t, a_idx])

# ##### END PART (a) #####

# Record Q-values for this epoch
Q_epoch[k] = Q

# Do value iteration
converged = False
eps = 1e-4
max_iters = 500
Q_vi = np.zeros((S.size, A.size))
Q_vi_prev = np.full(Q_vi.shape, np.inf)

for k in tqdm(range(max_iters)):

    # ##### PART (b): YOUR CODE BELOW #####

    Q_vi_new = np.zeros_like(Q_vi)
    for i, s in enumerate(S):
        for j, a in enumerate(A):
            Q_vi_new[i, j] = sum(
                P[l] * (reward(s, a, d) +  $\gamma$  * np.max(Q_vi[transition(s, a, d)]))
                for l, d in enumerate(D)
            )
    Q_vi_prev = Q_vi.copy()
    Q_vi[:] = Q_vi_new

    # ##### END PART (b) #####

    if np.max(np.abs(Q_vi - Q_vi_prev)) < eps:
        converged = True
        print("Value iteration converged after {} iterations.".format(k))
        break
    else:
        np.copyto(Q_vi_prev, Q_vi)

if not converged:
    raise RuntimeError("Value iteration did not converge!")

# Plot Q-values for each epoch
fig, axes = plt.subplots(
    2, S.size // 2, figsize=(12, 6), sharex=True, sharey=True, dpi=150
)
fig.subplots_adjust(hspace=0.2)
for i, ax in enumerate(axes.ravel()):
    for j in range(A.size):
        plot = ax.plot(Q_epoch[:, i, j], label="$a = {}".format(A[j]))
        ax.axhline(Q_vi[i, j], linestyle="--", color=plot[0].get_color())
        ax.legend(loc="lower right")
        ax.set_title(r"$s = {}".format(S[i]))
for ax in axes[-1, :]:
    ax.set_xlabel("epoch")
for ax in axes[:, 0]:
    ax.set_ylabel("$Q(s,a)$")
fig.savefig("widget_sales_qvalues.png", bbox_inches="tight")
plt.show()

# Report optimal policies from Q-learning and value iteration, simulate them
# for 5 years, and plot the cumulative profits

# ##### PART (c): YOUR CODE BELOW #####

# INSTRUCTIONS: Compute the optimal actions `a_opt_ql` and `a_opt_vi` using the
# Q-values from Q-learning and value iteration, respectively.
# Both `a_opt_ql` and `a_opt_vi` should be `np.ndarray`s, where
# each entry is the optimal action for the corresponding state.
#
# Also, simulate each optimal policy and compute the history of
# cumulative profits `profit_ql` and `profit_vi` over 5 years
# (at 365 days per year).

T = 5 * 365

a_opt_ql = A[np.argmax(Q, axis=1)]

```

```

a_opt_vi = A[np.argmax(Q_vi, axis=1)]

_, _, r_ql = simulate(rng, lambda s: a_opt_ql[int(s)], T)
profit_ql = np.cumsum(r_ql)

_, _, r_vi = simulate(rng, lambda s: a_opt_vi[int(s)], T)
profit_vi = np.cumsum(r_vi)

# ##### END PART (c) #####

print("Optimal policy (Q-learning):", a_opt_ql)
print("Optimal policy (value iteration):", a_opt_vi)

fig, ax = plt.subplots()
ax.plot(profit_ql, label=r"$Q$-learning")
ax.plot(profit_vi, label=r"value iteration")
ax.legend(loc="lower right")
ax.set_xlabel(r"day $t$")
ax.set_ylabel(r"cumulative profit $\sum_{k=0}^t r_k$")
fig.savefig("widget_sales_profits.png", bbox_inches="tight")
plt.show()

```

agents/qlearning.py

```

from .Agent import Agent, Transition
from collections import deque
from typing import Union
import gymnasium as gym
from tqdm import tqdm
import numpy as np
import math, torch, random

class QLearning(Agent):
    def __init__(self, state_dim : int, action_dim : int, hidden_dim : int=24, use_gpu : bool=False) -> None:
        super().__init__(state_dim, action_dim, hidden_dim, use_gpu)
        self.policy_network = self.build_network().to(self.device)
        self.buffer = deque([], maxlen=1000) # empty replay buffer with the 1000 most recent transitions
        self.agent_name = "qlearning"

        self.iteration = 0

    def eps_threshold(self) -> float: # epsilon threshold for e-greedy exploration
        eps_start, eps_end, eps_decay = 0.9, 0.05, 1000
        self.iteration += 1
        return eps_end + (eps_start - eps_end) * math.exp(-1 * self.iteration / eps_decay)

    def build_network(self) -> torch.nn.Module:
        """ WRITE YOUR CODE BELOW """
        """ 1) Construct and return a Multi-Layer Perceptron (MLP) representing the Q function. Recall th
at a Q function accepts
        """ two arguments i.e., a state and action pair. For this implementation, your Q function will pr
ocess an observation
        """ of the state and produce an estimate of the expected, discounted reward for each available ac
tion as an output -- allowing you to
        """ select the prediction associated with either action.
        """ 2) Use a hidden layer dimension as specified by 'self.hidden_dim'.
        """ 3) Our solution implements a three layer MLP with ReLU activations on the first two hidden un
its.
        """ But you are welcome to experiment with your own network definition!
        """
        """ Please see the following docs for support:
        """ nn.Sequential: https://docs.pytorch.org/docs/stable/generated/torch.nn.Sequential.html
        """ nn.Linear: https://docs.pytorch.org/docs/stable/generated/torch.nn.Linear.html
        """ nn.ReLU: https://docs.pytorch.org/docs/stable/generated/torch.nn.ReLU.html
        Q=torch.nn.Sequential(
            torch.nn.Linear(self.state_dim, self.hidden_dim),
            torch.nn.ReLU(),
            torch.nn.Linear(self.hidden_dim, self.hidden_dim),
            torch.nn.ReLU(),
            torch.nn.Linear(self.hidden_dim, self.action_dim)
        )
        return Q
        """

    def policy(self, state : Union[np.ndarray, torch.tensor], train : bool=False) -> torch.Tensor:
        if isinstance(state, np.ndarray):
            state = torch.tensor(state).unsqueeze(0).to(self.device)
        """ WRITE YOUR CODE BELOW """
        """ 1) If train == True, sample from the policy with e-greedy exploration with a decaying epsilon
threshold. We've already
        """ implemented a function you can use to call the exploration threshold at any instance in the i
teration i.e., 'self.eps_threshold()'.
        """ 2) If train == False, sample the action with the highest Q value as predicted by your networ
k.
        """
        """ HINT: An exemplar implementation will need to use torch.no_grad() in the solution.
        """
        """ Please see the following docs for support:
        """ random.random: https://docs.python.org/3/library/random.html#random.random
        """ torch.randint: https://docs.pytorch.org/docs/stable/generated/torch.randint.html
        """ torch.argmax: https://docs.pytorch.org/docs/stable/generated/torch.argmax.html
        """ torch.no_grad(): https://docs.pytorch.org/docs/stable/generated/torch.no_grad.html
        if train:
            if random.random() < self.eps_threshold():
                return torch.randint(0, self.action_dim, (1,)).to(self.device)
            else:
                with torch.no_grad():
                    return torch.argmax(self.policy_network(state)).unsqueeze(0)
        else:
            with torch.no_grad():
                return torch.argmax(self.policy_network(state)).unsqueeze(0)
        """

```

```

def sample_buffer(self) -> tuple[torch.Tensor, torch.Tensor]:
    assert len(self.buffer) > self.batch_size
    samples = random.sample(self.buffer, self.batch_size)

    states, actions, targets = [], [], []
    for i in range(self.batch_size):
        s, a, r, sp = samples[i].state.to(self.device), samples[i].action.item(), samples[i].reward.to(self.device), samples[i].next_state if samples[i].next_state is None else samples[i].next_state.to(self.device)
        states.append(s)
        actions.append(a)
        with torch.no_grad():
            targets.append(r if sp is None else r + self.gamma*torch.max(self.policy_network(sp)))

    return torch.cat(states), torch.tensor(actions, dtype=torch.int64).to(self.device).unsqueeze(1), torch.cat(targets).unsqueeze(1)

def train(self, env : gym.wrappers, num_episodes : int=100) -> None:
    ### WRITE YOUR CODE BELOW #####
    ### 1) Implementing the training algorithm according to Algorithm 1 on page 5 in "Playing Atari with Deep Reinforcement Learning".
    ### 2) Importantly, only take a gradient step on your memory buffer if the buffer size exceeds the batch size hyperparameter.
    ### HINT: In our implementation, we used the AdamW optimizer.
    ### HINT: Use the custom 'Transition' data structure to push observed (s, a, r, s') transitions to the memory buffer. Then,
    ### you can sample from the buffer simply by calling 'self.sample_buffer()'.
    ### HINT: In our implementation, we clip the value of gradients to 100, which is optional.
    ### Please see the following docs for support:
    ### torch.optim.AdamW: https://docs.pytorch.org/docs/stable/generated/torch.optim.AdamW.html
    ### torch.nn.MSELoss: https://docs.pytorch.org/docs/stable/generated/torch.nn.MSELoss.html
    ### torch.nn.utils.clip_grad_value_: https://docs.pytorch.org/docs/stable/generated/torch.nn.utils.clip\_grad\_value\_.html
    optimizer=torch.optim.AdamW(self.policy_network.parameters(), lr=self.learning_rate)
    loss_fn=torch.nn.MSELoss()
    for episode in tqdm(range(num_episodes)):
        state, _ = env.reset()
        done = False
        while not done:
            action=self.policy(state, train=True)
            next_state, reward, terminated, truncated, _ = env.step(action.item())
            done=terminated or truncated
            sp = None if terminated else torch.tensor(next_state).unsqueeze(0).to(self.device)
            self.buffer.append(Transition(torch.tensor(state).unsqueeze(0).to(self.device), action, torch.tensor(reward).unsqueeze(0).to(self.device), sp))
            state=next_state

        if len(self.buffer)>self.batch_size:
            states, actions, targets=self.sample_buffer()
            predictions=self.policy_network(states).gather(1, actions)
            loss=loss_fn(predictions, targets)
            optimizer.zero_grad()
            loss.backward()
            torch.nn.utils.clip_grad_value_(self.policy_network.parameters(), 100)
            optimizer.step()
    #####

```

agents/ReinforceAgent.py

```

from .Agent import Agent
from typing import List
import gymnasium as gym
import numpy as np
import matplotlib.pyplot as plt
import torch, datetime

class ReinforceAgent(Agent):
    def __init__(self, state_dim : int, action_dim : int, hidden_dim : int=24) -> None:
        super().__init__(state_dim, action_dim, hidden_dim)
        self.policy_network = self.build_network()
        self.optimizer = torch.optim.Adam(self.policy_network.parameters())
        self.agent_name = "reinforce"

    def build_network(self) -> torch.nn.Module:
        return torch.nn.Sequential(
            torch.nn.Linear(self.state_dim, self.hidden_dim),
            torch.nn.ReLU(),
            torch.nn.Linear(self.hidden_dim, self.action_dim),
            torch.nn.Softmax(dim=-1)
        )

    def policy(self, state: np.ndarray, train : bool=False) -> int:
        state = torch.tensor(state, dtype=torch.float)
        action_probs = self.policy_network(state)
        action_distribution = torch.distributions.Categorical(action_probs)
        action = action_distribution.sample()
        if train:
            log_prob = action_distribution.log_prob(action)
            return action, log_prob
        else:
            return action

    def train(self, env: gym.wrappers, num_episodes: int=500) -> None:
        reward_history = []
        for episode in range(num_episodes):
            obs, info = env.reset(seed=1738)
            terminated, truncated = False, False

            log_probs = []
            rewards = []

            while not terminated and not truncated:
                action, log_prob = self.policy(obs, train=True)
                obs, reward, terminated, truncated, info = env.step(action.item())
                log_probs.append(log_prob)
                rewards.append(reward)

            self.learn(rewards, log_probs)
            total_reward = sum(rewards)
            reward_history.append(total_reward)
            print(f"Episode {episode+1}: Total Reward = {total_reward}")

        self.plot_rewards(reward_history)

    def learn(self, rewards: list, log_probs: list) -> None:
        """ WRITE YOUR CODE BELOW """
        """ 1) Implement the naive REINFORCE, REINFORCE with causality trick and REINFORCE with causality
        trick + a baseline to 'center' the returns. """
        """ 2) After you've finished your implementation, please comment out all sections but the section
        you wish to evaluate for training. """
        """ Please see the following docs for support:
        torch.stack: https://docs.pytorch.org/docs/stable/generated/torch.stack.html """

        T = len(rewards)
        log_probs = torch.stack(log_probs)

        # # 1) Naive REINFORCE
        # R = sum(self.gamma**t * rewards[t] for t in range(T))
        # loss = -(log_probs * R).sum()

        # # 2) REINFORCE with causality trick
        # returns = torch.zeros(T)
        # for t in range(T):
        #     returns[t] = sum(self.gamma**(tp - t) * rewards[tp] for tp in range(t, T))
        # loss = -(log_probs * returns).sum()

        # 3) REINFORCE with causality trick and baseline to "center" the returns

```

```

returns = torch.zeros(T)
for t in range(T):
    returns[t] = sum(self.gamma**(tp - t) * rewards[tp] for tp in range(t, T))
baseline = sum(self.gamma**t * rewards[t] for t in range(T)) / T
loss = -(log_probs * (returns - baseline)).sum()
#####

self.optimizer.zero_grad()
loss.backward()
self.optimizer.step()

@staticmethod
def plot_rewards(reward_history: List[int]) -> None:
    current_time = datetime.datetime.now().strftime("%Y%m%d_%H%M%S")

    plt.figure()
    plt.plot(reward_history)
    plt.xlabel('Episode')
    plt.ylabel('Total Reward')
    plt.title('Training Reward Curve')
    filename = f"reward_curve_{current_time}.png"
    plt.savefig(filename)
    plt.show()
    print(f"Saved reward curve as {filename}")

```